Finding Similar Scientific Abstracts. Algorithm from Scratch

Course "Algorithms for Massive Data"

Master in Data Science for Economics, Università degli Studi di Milano

Nikita Olefir | Matricola 37003A | Academic Year 2025/26

Contents

1 Introduction **2**

2 Dataset **2**

3 Data Organisation and Preprocessing **2**

3.1 Baseline using word sets . . . 2

3.2 Shingling . . . 3

4 Algorithms **4**

4.1 Jaccard Similarity . . . 4

4.2 MinHashing . . . 4

4.3 Locality-Sensitive Hashing . . . 5

4.4 Distance Measures . . . 5

5 Experiments **6**

5.1 Why Brute Force Does Not Scale . . . 6

5.2 Full Pipeline on 50k Documents . . . 6

5.3 Results . . . 7

6 Scalability and the MapReduce Model **7**

7 Discussion **8**

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This project finds pairs of similar scientific paper abstracts in a large corpus. The arXiv dataset has roughly 3 million papers spanning mathematics, physics, computer science, and other fields. Comparing all pairs in even a 50 000 document might be a long task, so I take a sample of articles. This still allows me to check the work of algorithms and implement them from scratch. In the end I discuss how MapReduce can be applied to lower the waiting time and run algorithms on the whole corpus.

I followed the standard pipeline from the course. Jaccard similarity is the measure of similarity I use throughout the project. Each abstract becomes a set of character n-grams called shingles. I compress those sets into compact MinHash signatures and then use Locality-Sensitive Hashing (LSH) to find candidate pairs without scanning every combination.

2 Dataset

I used the arXiv metadata snapshot published by Cornell University on Kaggle.[1] Each line in the JSONL file is a paper record with fields `id`, `title`, `abstract`, and `categories`. The full file has approximately 3 million records.

I loaded the first 50 000 papers. That is enough to make brute force comparison impractical while keeping runtimes manageable on a single machine. Table 1 summarises the abstract lengths.

Statistic	Count	Mean	Std	Min	Max
Abstract length (chars)	50 000	789.9	402.0	17	2861

Table 1: Summary statistics for abstract character lengths.

Abstracts vary a lot in length, from a handful of words to nearly 3 000 characters. Most papers fall in the 500 to 1000 character range.

3 Data Organisation and Preprocessing

3.1 Baseline using word sets

I started by representing each abstract as a bag of words. Split on whitespace, lowercase everything, keep unique terms as a set.

Figure 1 shows the pairwise similarity matrix on a sample of 100 abstracts. Most values are near zero. Papers from different fields share very little vocabulary.

[1] https://www.kaggle.com/datasets/Cornell-University/arxiv

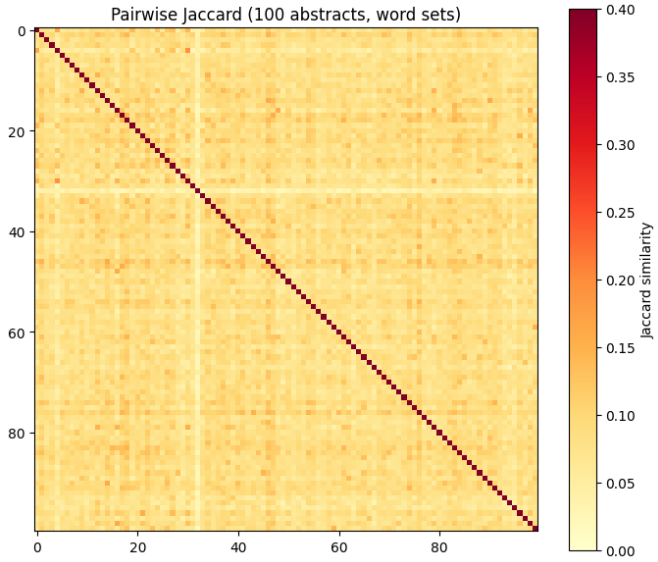

Figure 1: Pairwise Jaccard similarity on word sets, 100 randomly sampled abstracts.

Figure 2 shows how similarities are distributed. Most pairs score below 0.05. Only a handful exceed 0.1.

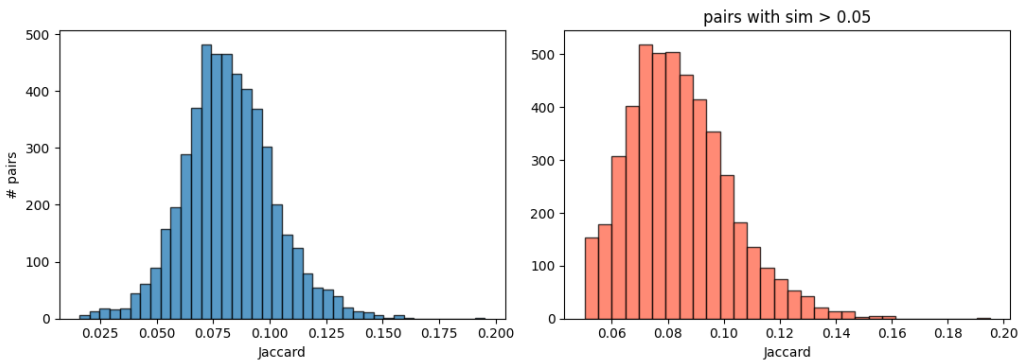

Figure 2: Distribution of pairwise Jaccard similarities using word sets. Left shows all pairs, right shows pairs with similarity above 0.05.

Word sets are a weak representation. They ignore word order and get confused by paraphrasing. Two abstracts on the same topic but written differently will share vocabulary and look similar even when the actual text differs.

3.2 Shingling

A better approach is to represent each document as a set of overlapping character substrings of length k, called shingles. Two documents that share phrases will share many shingles. Documents that just use similar vocabulary but phrase things differently will not.

The shingle set of a document d with parameter k is

$$\mathcal{S}_k(d) = \{d[i : i + k] \mid 0 \leq i \leq |d| - k\}$$

where d is the lowercased and whitespace normalised version of the abstract. I hash each shingle to a 32-bit integer using Python's built-in `hash()` function to avoid storing raw strings. This cuts memory by roughly 30× without changing similarity values, as verified experimentally.

I tested $k \in \{3, 5, 7, 9\}$ on a small sample. Average shingle set sizes were 438, 607, 672, and 703. Small k creates too many collisions between unrelated documents. Very large k means similar documents may not share enough shingles. I settled on $k = 9$. At that size, shingles are specific enough to mean something but forgiving enough to survive minor paraphrasing.

Shingles are stricter than word sets. Papers from different fields often share technical phrases like "we show" or "let G be a graph", which inflates word set similarity. Any real shingle overlap is a much stronger signal of genuine textual similarity.

4 Algorithms

4.1 Jaccard Similarity

The Jaccard similarity between two sets S and T is

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

It equals 1 for identical sets and 0 for disjoint ones. The Jaccard distance $d(S, T) = 1 - J(S, T)$ satisfies all metric axioms and works as a proper distance measure.

4.2 MinHashing

Computing exact Jaccard for 50 000 documents means evaluating $|S \cap T|$ and $|S \cup T|$ for roughly 1.25 billion pairs. That is too slow regardless of how the shingle sets are stored. MinHashing gives a compact approximation instead.

For a uniformly random permutation π of all possible shingle values, the probability that the minimum element of $\pi(S)$ equals the minimum element of $\pi(T)$ is exactly $J(S, T)$

$$P[\min \pi(S) = \min \pi(T)] = J(S, T)$$

Simulating n independent random permutations is done using linear hash functions

$$h_i(r) = (a_i \cdot r + b_i) \bmod p$$

Here p is a large prime (I use $p = 2^{31} - 1$) and a_i, b_i are random. The signature of a document is the vector of minimum hash values across all its shingles, one per hash function. Matching entries across n functions gives an unbiased estimate of the Jaccard similarity.

Table 2 shows how error changes with the number of hash functions.

Hash functions	MAE
10	0.0120
25	0.0084
50	0.0061
100	0.0042
200	0.0037

Table 2: Mean absolute error of MinHash estimates vs. true Jaccard, on 190 document pairs.

Error shrinks roughly as $1/\sqrt{n}$ but gains diminish past 100. I used $n = 100$ hash functions, producing a $100 \times 50\,000$ signature matrix of about 40 MB.

4.3 Locality-Sensitive Hashing

Even with compact signatures, comparing all pairs is still $O(n^2)$. LSH fixes this by using the structure of the signature matrix to find candidates without exhaustive comparison.

I split the signature matrix into b bands of r rows each, where $b \cdot r = n_{\text{hashes}}$. Two documents become candidates if they match exactly in at least one band. For a pair with true Jaccard similarity s, the probability of becoming a candidate is

$$P(\text{candidate} \mid s) = 1 - (1 - s^r)^b$$

This gives an S shaped curve that rises steeply around a threshold $t \approx (1/b)^{1/r}$. Pairs below the threshold are mostly ignored and pairs above are mostly flagged. Figure 3 shows the S curves for several (b, r) configurations.

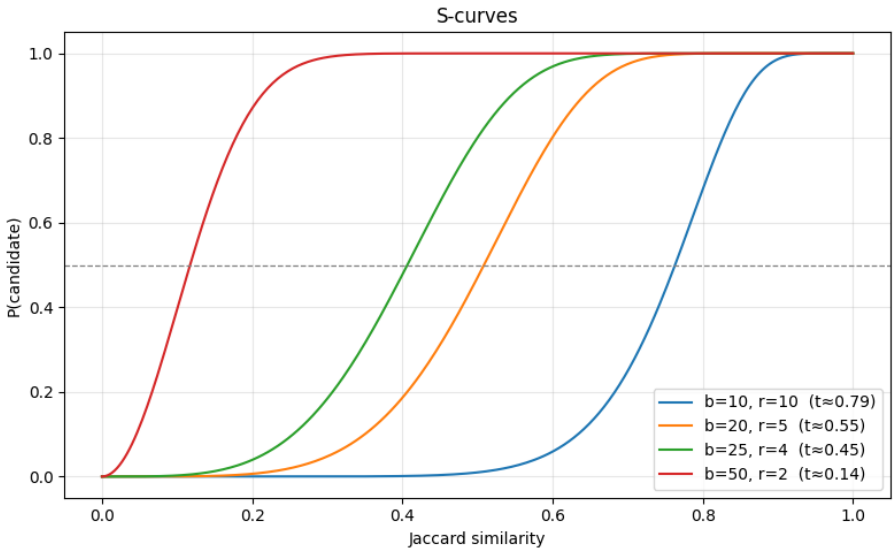

Figure 3: LSH S curves for four (b, r) configurations. The dashed line marks $P = 0.5$.

The choice of (b, r) trades precision for recall.

- Small r, large b gives a low threshold. More candidates, higher recall, but also more false positives.
- Large r, small b raises the threshold. Fewer candidates, higher precision, but some pairs may be missed.

I used $b = 25$, $r = 4$, which sets the threshold at roughly 0.45. Then I filtered candidates by requiring an estimated similarity of at least 0.5. LSH casts a slightly wide net and the final signature check removes the borderline cases.

4.4 Distance Measures

I also implemented several other distance measures to understand how they compare.

Jaccard distance. $d_J(S, T) = 1 - J(S, T)$ is the direct complement of Jaccard similarity. This is the main distance used throughout.

Cosine distance. For documents represented as TF vectors x and y, cosine similarity is

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \cdot \|y\|}$$

and cosine distance is $d(x, y) = 1 - \mathrm{sim}(x, y) \in [0, 1]$. For unrelated abstracts this sits around 0.5, which is consistent with high-dimensional sparse vectors.

L_p distances. For a TF vector $x = (x_1, \ldots, x_n)$, the L_p norm is $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$. The L_1 (Manhattan) and L_2 (Euclidean) cases give count-based distances between documents. They are less useful here because abstract lengths vary a lot, inflating distances for longer documents.

Edit distance. Let $\mathrm{LCS}(s, t)$ denote the length of the longest common subsequence of strings s and t. Edit distance is then

$$\mathrm{edit}(s, t) = |s| + |t| - 2 \cdot \mathrm{LCS}(s, t)$$

computed via dynamic programming in $O(|s| \cdot |t|)$ time. On typical abstracts this is too slow at scale, but it gives exact values and is useful as ground truth on small samples.

5 Experiments

5.1 Why Brute Force Does Not Scale

With 50 000 documents there are $\binom{50000}{2} \approx 1.25 \times 10^9$ pairs. Each exact Jaccard comparison takes about a microsecond, so brute force takes around 20 minutes total. That is slow but manageable. At 1 million documents it takes nearly 6 days. At the full 3 million it simply does not work on a laptop.

n documents	pairs	estimated time
100	4 950	$<$ 1s
1 000	499 500	$<$ 1s
10 000	49 995 000	∼50s
50 000	1 249 975 000	∼20.8 min
1 000 000	499 999 500 000	∼5.8 days

Table 3: Brute force scaling. Pairs and estimated computation time at 10^{-6} s per comparison.

5.2 Full Pipeline on 50k Documents

Table 4 shows the full pipeline timing on 50 000 documents.

Step	Time	Output
Shingling ($k = 9$, hashed)	7.3s	50 000 shingle sets
MinHashing (100 functions)	78.9s	$100 \times 50\,000$ signature matrix
LSH ($b = 25$, $r = 4$)	13.6s	2 379 candidate pairs
Signature filter (≥ 0.5)	0.0s	1 196 similar pairs
Total	∼100s	

Table 4: Full pipeline timing on 50 000 abstracts.

LSH reduces the search space from 1.25 billion pairs to just 2 379 candidates, a reduction of over 99.9998%. Only 0.0002% of all pairs reach the final verification step.

MinHashing dominates the runtime at 79 seconds out of 100 total. The current implementation loops over documents in Python and calls NumPy row by row. A fully vectorised implementation would be much faster.

5.3 Results

Figure 4 shows the similarity distribution among the 1 196 detected pairs and compares MinHash estimates to true Jaccard values.

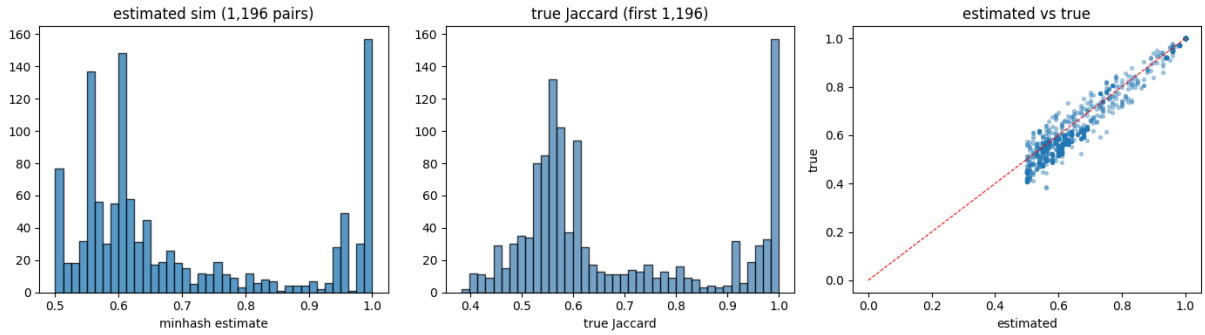

Figure 4: Left shows the distribution of MinHash similarity estimates for detected pairs. Centre shows the true Jaccard distribution. Right compares estimated vs. true Jaccard, with the red dashed line at $y = x$.

The distribution is bimodal. Many pairs cluster near similarity 1.0 (duplicate or near duplicate papers) and a smaller group falls in the 0.5 to 0.8 range. The mean absolute error between MinHash estimates and true Jaccard is 0.033. This is higher than the 0.004 seen on random pairs, but expected. The filter selects high similarity pairs where estimation variance matters more.

Looking at the top results, three patterns stand out.

- **True duplicates.** The same paper submitted more than once, sometimes with a slightly different title. For example, "Some series and integrals involving the Riemann zeta function" appears four times in the top results with identical abstracts and similarity 1.0.
- **Version pairs.** Earlier and later versions of the same preprint. One example is "Relativistic treatment in D-dimensions to a spin-zero particle" paired with "Elativistic treatment in D-Dimensions to a spin-zero particle" (a typo in the title of the revised version), with true Jaccard 1.0.
- **Related work.** Papers on the same narrow topic from the same group, sharing boilerplate phrasing. The pair "William H. Kruskal and the Development of Coordinate-Free Methods" and "William Kruskal Remembered" are two separate tributes with overlapping text.

The algorithm also catches some false positives with similarity 1.0 where the titles are clearly unrelated, like a cryptography paper paired with an astrophysics paper. These are hashing collisions in the 32 bit hash space. With 100 hash functions and 2^{32} possible values this is rare but it does happen.

6 Scalability and the MapReduce Model

The pipeline runs sequentially on a single machine, but each step maps naturally onto the MapReduce model.

Stage 1. Shingling and MinHashing (pure Map). Each document is shingled and signed independently. A Map over all documents emitting `(doc_id, signature)` needs no Reduce step at all. This stage scales linearly.

Stage 2. LSH banding (Map + Reduce). For each band b and each document, emit `((b, band_hash), doc_id)`. A Reduce groups all documents in the same bucket and emits candidate pairs. Each Reduce touches only one bucket, so memory per reducer stays small.

Stage 3. Candidate verification (pure Map). Each candidate pair is verified independently by computing the exact Jaccard of the shingle sets. No Reduce step is needed.

Apache Spark RDD operations map directly onto these three stages. The main bottleneck appears at the boundary between stage 1 and stage 3. In a naive implementation, shingle sets are computed in stage 1 and then discarded. Stage 3 then has to recompute them from scratch to verify candidate pairs. On 50 000 documents shingling takes 6.7 seconds, so this is not dramatic, but at 3 million documents it adds up to roughly 400 seconds of redundant work. Keeping shingle sets in memory across stages avoids this entirely. The shingling and verification steps also parallelize naturally on a single machine using Python's `multiprocessing` module, which cuts the runtime substantially on a multicore machine.

7 Discussion

The pipeline works well. Here are the main takeaways.

- **Shingles outperform word sets.** Word sets are too noisy for detecting near duplicate text. Any real shingle overlap is a much stronger signal.
- **100 hash functions is enough.** The MAE is 0.004, and bumping to 200 only saves another 0.0005. Given a final threshold of 0.5, that level of noise is fine.
- **The LSH configuration matters.** With $b = 25$ and $r = 4$, the threshold sits at roughly 0.45. Combined with a post filter at 0.5, this gives a good balance between precision and recall. A higher threshold like $b = 10$, $r = 10$ (where $t \approx 0.79$) finds only near exact duplicates. A lower threshold floods the candidate set.
- **MinHashing is the bottleneck.** It takes 79 of the 100 total seconds. Scaling to 3M documents would take roughly 80 minutes for this step alone on a single core. A vectorised implementation or distributed execution fixes this.
- **Some false positives are hash collisions.** A few detected pairs with similarity 1.0 have clearly unrelated titles. These are 32 bit hash collisions in the shingle sets. Switching to 64 bit hashes reduces this substantially.